AI-POWERED AGENTIC HIRING PLATFORM

A MAJOR PROJECT REPORT

by

ABHIJITH BINOSH (VJC22AD001)

JOSE MARTIN BINU (VJC22AD038)

R JAYADEV (VJC22AD055)

SURYANARAYANAN N J (VJC22AD063)

in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

in

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

APJ ABDUL KALAM TECHNOLOGICAL UNIVERSITY

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

VISWAJYOTHI COLLEGE OF ENGINEERING AND TECHNOLOGY, VAZHAKULAM

APRIL 2026

AI-POWERED AGENTIC HIRING PLATFORM

MAJOR PROJECT REPORT

by

ABHIJITH BINOSH (VJC22AD001)

JOSE MARTIN BINU (VJC22AD038)

R JAYADEV (VJC22AD055)

SURYANARAYANAN N J (VJC22AD063)

in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

in

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

APJ ABDUL KALAM TECHNOLOGICAL UNIVERSITY

Under the guidance of

MRS. JOBY ANU MATHEW

ASSISTANT PROFESSOR

DEPT OF AI & DS, VJCET

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

VISWAJYOTHI COLLEGE OF ENGINEERING AND TECHNOLOGY, VAZHAKULAM

APRIL 2026

VISWAJYOTHI COLLEGE OF ENGINEERING AND TECHNOLOGY, VAZHAKULAM

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

BONAFIDE CERTIFICATE

This is to certify that the major project report entitled **'AI-POWERED AGENTIC HIRING PLATFORM'** is a bonafide record of the project presented by **ABHIJITH BINOSH (VJC22AD001)**, **JOSE MARTIN BINU (VJC22AD038)**, **R JAYADEV (VJC22AD055)**, and **SURYANARAYANAN N J (VJC22AD063)**, in partial fulfillment of the requirements for the award of the Degree of Bachelor of Technology in Artificial Intelligence and Data Science of APJ Abdul Kalam Technological University.

Project Guide

Project Co-ordinator

External Evaluator

Head of the Department

VISWAJYOTHI COLLEGE OF ENGINEERING AND TECHNOLOGY, VAZHAKULAM

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

Vision

Moulding professionals catering to research, innovation, and entrepreneurial developments for global digitalization.

Mission

1. To inculcate research culture for design and innovative development towards a better world.
2. To guide and mould the students to become globally competent professionals with ethical values.
3. To generate innovative ideas, and disseminate it through quality publications.

Program Educational Objectives

Our Graduates

1. Shall apply their knowledge from undergraduate education to adapt with the recent technological advancements and acquire a promising career.
2. Shall possess critical thinking, professional skills, and strong work ethics to solve real-world problems catering to both industry needs and societal demands.
3. Shall learn, apply, and adapt to grow with industrial needs thereby becoming successful innovators in the challenging technological world.
4. Shall have the competency to seek higher professional degrees, certifications, and to do quality research as an individual or as a team.

Program Outcomes

1. **Engineering knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
2. **Problem analysis:** Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles.
3. **Design / development of solutions:** Design solutions for complex engineering problems and system components that meet specified needs with appropriate consideration for public health, safety, and societal requirements.
4. **Conduct investigations of complex problems:** Use research-based knowledge and methods including design of experiments, data analysis, and synthesis of information to provide valid conclusions.
5. **Modern tool usage:** Create, select, and apply appropriate techniques, resources, and modern engineering tools including prediction and modelling to complex engineering activities.
6. **The engineer and society:** Apply reasoning informed by contextual knowledge to assess societal, health, safety, legal, and cultural issues relevant to professional engineering practice.
7. **Environment and sustainability:** Understand the impact of professional engineering solutions in societal and environmental contexts, and demonstrate the need for sustainable development.
8. **Ethics:** Apply ethical principles and commit to professional ethics, responsibilities, and norms of engineering practice.
9. **Individual and team work:** Function effectively as an individual, and as a member or leader in diverse teams and multidisciplinary settings.
10. **Communication:** Communicate effectively on complex engineering activities with the engineering community and society, including writing effective reports and making effective presentations.

11. **Project management and finance:** Demonstrate knowledge of engineering and management principles and apply these to manage projects in multidisciplinary environments.
12. **Life-long learning:** Recognize the need for and the ability to engage in independent and life-long learning in the broadest context of technological change.

Program Specific Outcomes

1. Able to apply the computational knowledge of data science, machine learning, and deep learning to analyse, design, and model novel applications.
2. Ability to become an entrepreneur which provides engineering solutions for industrial and societal problems.
3. Up-skilling in advanced data science, machine learning, and deep learning technologies.

ACKNOWLEDGEMENT

First and foremost, we thank God Almighty for His divine grace and blessings throughout the project, without which this work would not have reached successful completion. It is our privilege to express our sincere gratitude to the management and Principal of Viswajyothi College of Engineering and Technology, Vazhakulam, for providing us with the opportunity and resources to undertake this major project during the final year of our B.Tech degree course.

We are deeply thankful to our Head of the Department, **Dr. Anita Brigit Mathew**, Department of Artificial Intelligence and Data Science, for her constant support, encouragement, and guidance throughout the course of this work. We would like to express our sincere gratitude and heartfelt thanks to our project guide, **Mrs. Joby Anu Mathew**, Assistant Professor, Department of Artificial Intelligence and Data Science, for her invaluable guidance, timely feedback, and continuous support. Her mentorship and technical insights greatly contributed to the successful completion of this project.

We also extend our sincere thanks to our tutor, **Mrs. Mary Nirmala George**, for her encouragement and support throughout the project. We are equally grateful to all the faculty members and lab staff of the Department of Artificial Intelligence and Data Science for their assistance and motivation during the academic year.

Finally, we express our heartfelt thanks to our families and friends whose constant encouragement and support helped us stay motivated during the challenging phases of this project.

DECLARATION

We, the undersigned, hereby declare that the project report entitled **AI-POWERED AGENTIC HIRING PLATFORM**, submitted in partial fulfillment of the requirements for the award of the Degree of Bachelor of Technology of the APJ Abdul Kalam Technological University, is a bonafide record of original work carried out by us under the supervision of **Mrs. Joby Anu Mathew**, Assistant Professor, Department of Artificial Intelligence and Data Science, Viswajyothi College of Engineering and Technology. The ideas, findings, and conclusions presented in this document are our own. Where the work of others has been referenced or cited, appropriate acknowledgement has been given. We declare that we have adhered to th e principles of academic honesty and integrity throughout this work, and that this submission has not been used, in whole or in part, for any other degree, diploma, or similar award at this or any other institution.

Place: Vazhakulam

Date: _____

ABHIJITH BINOSH

JOSE MARTIN BINU

R JAYADEV

SURYANARAYANAN N J

ABSTRACT

The conventional hiring process is marked by high-volume manual resume screening, inconsistent evaluation, and considerable recruiter effort — challenges that grow more acute as online job platforms continue to attract larger applicant pools. This project presents the design and development of **HireX**, an AI-Powered Agentic Hiring Platform that automates the recruitment lifecycle through a multi-agent software architecture, bringing intelligence and transparency to each stage of the process. Rather than replacing human judgement entirely, the system structures and supports it — surfacing the most relevant candidates first, explaining why each candidate was ranked as they were, and handling the logistical mechanics of interview planning automatically.

At the core of the platform is an AI shortlisting engine that parses submitted PDF resumes using the `pdf-parse` library, constructs a unified candidate context from both structured profile data and extracted resume text, and computes a cosine similarity match score using TF-IDF vectorisation via the `natural` NLP library. Each score is accompanied by an explainable report that lists matched skills, missing skills, education match status, and a human-readable assessment — making the ranking auditable and actionable. Interview scheduling is handled by a break-aware round-robin algorithm that distributes candidate interviews equitably across interviewers while respecting configurable break intervals.

The platform separates job seekers and recruiters into distinct role-based workflows, each served by a dedicated set of features. Candidates can build structured profiles, upload multiple resume versions, and discover jobs through a unified interface that combines internal postings with live listings from the Adzuna and Jooble APIs. Recruiters can post jobs, manage applicants, trigger AI-driven shortlisting, and generate interview schedules — all within a single dashboard. The backend runs on Node.js with Express.js, data is managed through a cloud-hosted PostgreSQL instance on Neon, and the frontend is built in React 18 with Tailwind CSS.

CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
ATS	Applicant Tracking System
API	Application Programming Interface
BERT	Bidirectional Encoder Representations from Transformers
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
DFD	Data Flow Diagram
ER	Entity-Relationship
GNN	Graph Neural Network
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
IDF	Inverse Document Frequency
JSON	JavaScript Object Notation
JWT	JSON Web Token
ML	Machine Learning
MVC	Model-View-Controller
NER	Named Entity Recognition
NLP	Natural Language Processing
RBAC	Role-Based Access Control
REST	Representational State Transfer
SQL	Structured Query Language
SPA	Single Page Application
TF	Term Frequency
TF-IDF	Term Frequency – Inverse Document Frequency
UI	User Interface
UML	Unified Modelling Language
URL	Uniform Resource Locator

CHAPTER 1

INTRODUCTION

The growing scale of online job markets has made traditional recruitment methods increasingly difficult to sustain. When a single job posting routinely attracts several hundred applications, the ability of a recruiter to manually review each resume with equal care and consistency is fundamentally limited. The result is a process that often favours speed over thoroughness — one where qualified candidates may be overlooked simply because their resumes use different but equally valid terminology for the skills being sought. These inefficiencies carry real costs: delayed hiring cycles, inconsistent evaluation criteria, and missed opportunities on both sides of the hiring table.

Advances in Artificial Intelligence, particularly in Natural Language Processing and semantic text matching, now offer practical tools for addressing these limitations without removing human judgement from the decision. By automating the computationally intensive work of reading, parsing, and comparing large volumes of resume text against job requirements, intelligent systems can surface the most relevant candidates quickly — while still leaving the final hiring decision in human hands. The challenge lies in doing this in a way that is transparent, auditable, and fair.

This project is a response to that challenge. **HireX** — the AI-Powered Agentic Hiring Platform developed through this work — provides a complete recruitment environment in which job seekers and recruiters operate through role-specific dashboards, supported at each stage by intelligent automation. The platform employs a TF-IDF cosine similarity engine to score and rank candidates against job descriptions, produces per-candidate explainability reports that allow recruiters to understand and justify every shortlisting decision, and uses a break-aware round-robin scheduling algorithm to handle the logistical complexity of assigning interview slots. External job aggregation through the Adzuna and Jooble APIs further broadens the platform's reach, giving job seekers access to live listings alongside internally posted positions without needing to visit multiple job boards.

The system is built on a modern full-stack architecture: a React 18 frontend with Tailwind CSS, a Node.js and Express.js backend with JWT-based authentication,

and a cloud-hosted PostgreSQL database on Neon. Together, these components form a platform that is not only functional but extensible — designed from the outset to accommodate future enhancements such as deep learning-based matching, email notifications, and mobile support.

1.1 MOTIVATION

Recruitment workflows in most organisations today are a mixture of manual effort and partial automation. While job boards and applicant tracking systems have made it easier to collect and store applications, the actual evaluation of candidates still depends heavily on a recruiter reading individual resumes and making subjective assessments. The limitations of this approach become clear at scale: time pressure leads to shortcuts, cognitive biases influence decisions, and the lack of explicit evaluation criteria makes it difficult to ensure consistency across reviewers or even across different days of review by the same person.

Existing applicant tracking systems often attempt to address this through keyword filtering — automatically eliminating applications that do not contain specific terms from a predefined list. This approach introduces a different and arguably more harmful problem: it penalises candidates who describe their experience in semantically equivalent but textually different language. A developer skilled in "Node.js" who writes "server-side JavaScript" in their resume should not be ranked lower than a candidate who happens to use the exact search keyword. Nor should a data analyst who lists "statistical modelling" be filtered out of a search for "predictive analytics." These are not edge cases; they are the normal product of the natural variation in how professionals document their expertise across different industries, seniorities, and cultural backgrounds.

Beyond the screening inefficiency, the industry-wide reliance on human resume review at scale also raises questions of consistency and fairness. Research has documented the influence of candidate name, educational institution prestige, and formatting style on recruiter decisions made in seconds during initial scan. These biases are difficult to eliminate through training alone when the process itself requires dozens of rapid, undocumented judgements per day.

The design of HireX was motivated by the conviction that AI can meaningfully

address these problems — not by replacing human judgement entirely, but by restructuring the point at which human attention is applied. If AI can reliably surface the most relevant candidates first and explain why each was ranked as it was, a recruiter can invest their attention in evaluating the shortlist rather than constructing it. This shift preserves human decision-making authority while removing the inconsistency and cognitive load that comes with processing large application volumes manually.

1.2 OBJECTIVE

The objectives of this project are to design and implement an end-to-end recruitment platform with the following core capabilities:

(1) Automate the resume screening and candidate ranking process using TF-IDF vectorisation and cosine similarity, replacing keyword filtering with a semantically informed scoring approach that captures term meaning rather than literal string presence.

(2) Produce explainable AI outputs for every shortlisted candidate, including matched skills, missing skills, education match verdict, and a plain-language summary that allows a recruiter to audit and justify every shortlisting decision.

(3) Implement a role-based platform that separates job-seeker and recruiter workflows while maintaining strict data access boundaries enforced through JWT authentication and middleware-level role guards that operate at the route level without relying on client-side restrictions.

(4) Aggregate live job listings from external APIs alongside internal postings to provide job seekers with a unified job discovery experience that eliminates the need to visit multiple job boards manually.

(5) Automate interview scheduling using a break-aware round-robin algorithm that distributes candidate interviews equitably across available interviewers while inserting configurable rest intervals to prevent interviewer fatigue.

(6) Design the system with scalability and extensibility in mind, ensuring that future AI improvements — such as transformer-based semantic matching or LLM-assisted job description enrichment — can be integrated as service-

level replacements without requiring changes to the database schema, frontend design, or API contract.

1.3 SCOPE

The scope of the platform covers the complete hiring lifecycle from job posting to interview assignment. The system is designed to serve two distinct user roles — job seekers and recruiters — each with a dedicated set of features, and to integrate an autonomous AI layer that processes data on behalf of both roles at key decision points. The following functional domains fall within the scope of this project:

1. **Identity and access management:** User registration, login, JWT issuance, and role-based route protection for job seekers and recruiters, including a secondary role guard middleware that enforces endpoint-level access control independently of the token verification step.

2. **Candidate profile management:** Structured profiles with education qualifications, work experience, technical skills as a JSONB array, project descriptions, and multiple resume file versions stored as base64-encoded PDFs with user-defined version labels.

3. **Job management:** Full create, read, update, and delete operations for job postings, including free-text job descriptions, structured skill requirements, education expectations, custom screening questions, and posting status management (open, closed, deleted).

4. **Application tracking:** The complete application lifecycle from submission through shortlisting to final decision, with immutable profile snapshots captured at submission time and status updates communicated to candidates in real time through the tracker interface.

5. **AI-driven candidate evaluation:** Automated PDF resume parsing using `pdf-parse`, candidate context construction from structured and unstructured profile data, TF-IDF cosine similarity scoring using the `natural` NLP library, and per-candidate explainability reports covering matched skills, missing skills, and education alignment.

6. **Interview scheduling:** Break-aware round-robin slot allocation across one or more interviewers, with configurable interview slot duration, break frequency, and break duration, and a Fisher-Yates randomised initial interviewer order to ensure equitable distribution.

7. **External job aggregation:** Real-time job fetching from Adzuna and Jooble APIs with response normalisation, cross-source deduplication, and presentation within the platform's unified job discovery interface.

The scope explicitly excludes browser push notifications, email communication, mobile client applications, video interview integration, and large language model-based job description enhancement, all of which are identified as priorities for future development. The system is intended for deployment in a local or cloud-hosted server environment for use by small to medium-sized recruiting organisations or academic institutions managing campus placement processes.

CHAPTER 2

LITERATURE SURVEY

A review of existing literature was conducted to understand the current state of AI-driven recruitment, covering topics from intelligent resume screening and person-job fit estimation to fairness-aware ranking and skill gap analytics. The works surveyed span a wide range of methodological approaches — from classical machine learning techniques such as SVM and TF-IDF to modern deep learning architectures including BERT-based bi-encoders and graph neural networks. Table 2.1 presents a consolidated summary of twenty significant research contributions that collectively shaped the design rationale and technical choices made in this project.

Several recurring themes emerge across the literature. There is a clear trend away from brittle keyword-based matching toward semantically informed models that better capture the contextual meaning of candidate profiles. At the same time, concerns around explainability, fairness, and computational efficiency consistently surface as practical constraints — particularly for platforms intended to scale across large applicant pools. The proposed system addresses these concerns by selecting a TF-IDF cosine similarity approach that is inherently interpretable and computationally lightweight, while laying the groundwork for future integration of more powerful semantic models.

Table 2.1: *Literature Survey Summary*

Sl.	Paper Title	Methodology	Advantages	Disadvantages
1	Person-job fit estimation with Co-Attention Neural Networks [1]	mashRNN hierarchical encoding extracts semantic features from long documents. Co-Attention aligns job requirements with resume experience. GNNs model global recruiter behaviour. BiLSTM with attention computes similarity edges.	Leverages recruitment history; handles lengthy documents; attention mechanism highlights matched skills with strong interpretability.	High computational cost; requires large historical datasets; slower inference due to graph processing overhead.
2	AI-Powered Resume Screening using NLP and Machine Learning [2]	Grid-search tuned SVM with cross-validation. TF-IDF vectorisation into 5000 features. NLP preprocessing: tokenisation, stopword removal, lemmatisation.	Achieved 91.3% classification accuracy; computationally efficient; strong generalisation via cross-validation.	Lacks semantic context; high-dimensional sparse matrix; limited to predefined job role categories.

Continued on next page

Table 2.1 — *Continued from previous page*

Sl.	Paper Title	Methodology	Advantages	Disadvantages
3	Intelligent Resume Evaluation Tool Based on ML [3]	Weighted resume scoring (Skills: 30, Education: 20, etc.). Skill gap analysis using spaCy PhraseMatcher against curated industry skill lists. Regex-based contact extraction.	Provides actionable feedback and course recommendations; converts qualitative content to comparable scores; high precision for structured resumes.	Relies on static predefined skill lists; subjective weight assignment; sensitive to resume format and structure.
4	Resume Ranker: AI-Based Skill Analysis and Matching [4]	Contextual keyword extraction via KeyBERT. Synonym generation using Word2Vec. Cosine similarity matching between candidate vectors and job descriptions.	Captures semantic nuances; handles skill synonym variations; produces clear percentage-based scores.	High BERT computational overhead; complex multi-model stack; performance depends on training corpus quality.

Continued on next page

Table 2.1 — *Continued from previous page*

Sl.	Paper Title	Methodology	Advantages	Disadvantages
5	Streamlining Talent Acquisition: A Machine Learning Approach [5]	AES-256 encrypted data processing. Bias-reduced matching using data-driven skill criteria. NLP parsing to extract structured fields from unstructured resumes.	GDPR-compliant; reduces unconscious bias; scalable across high application volumes.	Encryption adds latency; requires robust key management; parsing accuracy varies with document structure.
6	Recruiter's Perception of AI-Based Tools in Recruitment [6]	Extended UTAUT framework with frequency of use and education variables. Mixed-method study with quantitative surveys. Web-based survey via Prolific Academic. Hierarchical regression analysis.	Holistic prediction model; transparent psychometric scales; reveals why recruiters value efficiency while fearing reduced human judgement.	Limited generalisability; self-report bias; demographic variables showed no significant moderating effect.

Continued on next page

Table 2.1 — *Continued from previous page*

Sl.	Paper Title	Methodology	Advantages	Disadvantages
7	Evolution of ATS: From Databases to Intelligent Platforms [7]	Comparative literature-based analysis. Embedding-based ranking with BERT/S-BERT dense vectors. Human-in-the-loop re-ranking. Governance matrix for bias audits.	Semantic precision; reduced time-to-fill; bias mitigation via inspectable trade-offs; event-driven architectures handle large applicant spikes.	Effectiveness depends on corpus quality; continuous bias monitoring increases overhead; historical inequities risk being encoded in new rankings.
8	Smart Hiring Redefined: An Intelligent Recruitment Management Platform [8]	J2EE three-tier architecture. MVC pattern separating business logic from UI. Role-based access control for students, enterprises, and admins. Black-box testing against requirements.	High service cohesion and low coupling; user-centric validation; normalised relational database ensures data integrity.	Service coupling reduces separation of concerns; struggles with high concurrency; lacks mobile adaptation.

Continued on next page

Table 2.1 — *Continued from previous page*

Sl.	Paper Title	Methodology	Advantages	Disadvantages
9	Enhancing Recruitment Efficiency: An Advanced ATS [9]	NLP using TF-IDF and tokenisation for resume parsing. K-Nearest Neighbours for skill proficiency classification. Agile/Scrum development model. Cloud-based deployment.	Effective structured data extraction; KNN pattern recognition; rapid feature increments via agile methodology.	KNN inefficient on large datasets; cloud infrastructure management overhead; integration complexity with legacy HR systems.
10	A Multi-Module ATS: From Job Posting to Performance Monitoring [10]	Modular MERN stack architecture. Separate employee management, hiring, and performance modules. Rule-based and ML scoring. Agile sprint-based development.	End-to-end lifecycle management; extensible modular design; reported >60% reduction in manual effort.	Limited generalisation across industries; performance modules may miss external qualitative factors; user feedback dependency may bias the system.

Continued on next page

Table 2.1 — *Continued from previous page*

Sl.	Paper Title	Methodology	Advantages	Disadvantages
11	Enhancing Talent Search Ranking with Role-Aware Expert Mixtures and LLM-based Fine-Grained JDs [11]	LLMs with Chain-of-Thought prompting for JD encoding. Gated Cross Network for JD-resume interactions. Role-Aware MMoE model. Multi-Task Learning predicting CTR, CVR, and relevance. Online A/B testing.	Captures signals missed by keyword matching; adapts to recruiter role heterogeneity; demonstrated 5.97% CVR and 17.29% CTR gains in production.	Relies on sufficient interaction history; does not model temporal dynamics; excess experts caused overfitting.
12	Fairness-Aware Ranking in Search & Recommendation Systems [12]	Bias quantification via Skew@k, MinSkew, MaxSkew, NDKL. DetGreedy score-maximising greedy mitigation. DetCons/DetRelaxed for fairness constraints. DetConstSort with interval constrained sorting.	Simple implementation; achieves equality of opportunity without significant business metric loss.	DetGreedy infeasible for ≥4 attribute values; NDKL cannot differentiate equal but opposite bias; Skew@k is sensitive to choice of k.

Continued on next page

Table 2.1 — *Continued from previous page*

Sl.	Paper Title	Methodology	Advantages	Disadvantages
13	AI-Driven Decision-Making System for Hiring Process [13]	Multi-agent pipeline: ingestion, context construction, verification, and scoring agents. Public data verification via MCP tools. Entity linking (NER+ED+NEL) for skill normalisation. Composite scoring with technical fit, culture fit, and risk penalties.	Reduces screening time from 3.33 to 1.70 hours per candidate; traceable component-level reports; estimated cost of $2.29 vs. $50–$95 for human recruiters.	Inherits biases from recruiter ground truth data; privacy concerns with public data scraping; operational costs tied to API pricing.
14	Optimizing Job Matching through Automated Resume Screening [14]	Preprocessing: removal of URLs, hashtags, punctuation. Tokenisation, stemming, lemmatisation. TF-IDF vectorisation. Comparison of KNN, Random Forest, SVM, and Logistic Regression.	SVM achieved ≈0.99 test accuracy; TF-IDF effectively highlights key terms; automates high-effort manual sorting.	KNN showed lowest accuracy; stemming less precise than lemmatisation; extensive preprocessing required before any model training.

Continued on next page

Table 2.1 — *Continued from previous page*

Sl.	Paper Title	Methodology	Advantages	Disadvantages
15	ResumAI: Revolutionising Resume Analysis with Multi-Model Intelligence [15]	Named Entity Recognition with SpaCy for contact extraction. PDF parsing with pdfminer. Cosine similarity on BERT and TF-IDF embeddings. Classification comparison: Naive Bayes, Linear SVC, k-NN.	Linear SVC achieved 98.63% accuracy; BERT captures contextual relationships; eliminates manual data crawling.	BERT is computationally expensive; Naive Bayes reached only 73.97% accuracy; full text vectorisation required prior to ML processing.
16	Predicting Skill Shortages in Labour Markets [16]	Data integration linking job ads with ANZSCO labour statistics. Revealed Comparative Advantage (RCA) metric for skill identification. XGBoost classification for shortage prediction. Lagged temporal features.	XGBoost achieved up to 83% Macro-F1; granular insight into emerging skills; scalable across advanced economies.	Auto-regressive inertia limits shortage prediction "flips"; job ads over-represent white-collar roles; class imbalance requires oversampling.

Continued on next page

Table 2.1 — *Continued from previous page*

Sl.	Paper Title	Methodology	Advantages	Disadvantages
17	A Comprehensive Survey of AI Techniques for Talent Analytics [17]	Taxonomy of AI applications in HR and labour markets. OCR and NLP for structured resume parsing. Deep models for person-job fit. Organisational Network Analysis via graph models. Time-series forecasting for skill demand.	BERT and GNNs capture rich contextual meaning; ONA reduces evaluation bias; efficiently automates resume screening; dynamically tracks evolving skills.	Requires large labelled HR datasets; risk of amplifying historical bias; ONA raises privacy concerns; black-box models limit explainability.
18	Tec-Habilidad: Skill Classification for Bridging Education and Employment [18]	Zero-shot extraction with GPT-3.5-Turbo. KSAO framework mapping into Knowledge, Skills, Abilities, and Other. Definition augmentation for context improvement. Zero-shot vs. fine-tuned classification benchmarking.	Addresses lack of Spanish skill datasets; definition addition significantly improved accuracy; fine-tuned Transformers outperformed human annotator agreement.	Zero-shot LLMs achieved only 0.40–0.48 accuracy vs. >0.85 for fine-tuned BERT; models struggled with soft skills; class overlap between Skill and Knowledge categories.

Continued on next page

Table 2.1 — *Continued from previous page*

Sl.	Paper Title	Methodology	Advantages	Disadvantages
19	Real-Time AI-Based Skill Gap Analysis and Adaptive Career Guidance [19]	NLP embeddings for candidate and job features. Weighted cosine similarity on skills and experience. Generative matching using Gemini Flash 1.5. Skill gap detection: critical vs. recommended. Real-time labour trend data integration.	Personalised project and course recommendations; holistic matching weighing verified and inferred skills; auto-updates profiles after assessments.	Weight tuning for similarity scoring is complex; heavy reliance on external APIs; input quality depends entirely on completeness of user profile data.
20	Using Graph Algorithms for Skills Gap Analysis [20]	Neo4j knowledge graph integrating employee data with O*NET ontology. PageRank to identify globally critical skills. Jaccard node similarity linking employees. Louvain community detection for functional groups. Triangle count for network cohesion.	O*NET provides standardised skill ontology; identifies hidden employee connections for internal mobility; personalised PageRank surfaces critical hiring skills.	Highly sensitive to the similarity cutoff threshold; large Louvain communities may be too generic; cold-start problem with sparse network data.

CHAPTER 3

PROPOSED SYSTEM

The proposed system, HireX, addresses the inefficiencies of conventional recruitment by bringing together multiple intelligent sub-systems under a unified, role-aware platform. Rather than building an isolated tool for one part of the hiring process, the design takes an end-to-end view — covering everything from job discovery and application submission through AI-driven candidate evaluation to structured interview scheduling. Each stage of the pipeline is handled by purpose-built components that communicate through a RESTful API layer, ensuring that the platform can support the distinct needs of job seekers and recruiters without one workflow interfering with the other.

The system's multi-agent personality manifests in the way its backend services operate: the resume parsing agent, the TF-IDF matching engine, the explainability generator, and the scheduling algorithm each execute as independent, stateless functions that can be invoked on demand and tested in isolation. This design keeps the platform lightweight while remaining extensible — future AI improvements can be introduced as drop-in replacements for individual service functions without requiring changes elsewhere.

3.1 PROCESS OVERVIEW

The hiring workflow in HireX proceeds through two parallel tracks that eventually converge at the point of candidate evaluation. Figure 3.1 illustrates this flow in full.

Recruiter workflow:

1. The recruiter registers and sets up a verified company profile with name, industry, and logo.
2. A job posting is created with a full description, required skills, expected education level, and optional custom screening questions.
3. Applications from job seekers accumulate in the applicant management dashboard as candidates submit their profiles.

4. The recruiter triggers the AI Auto-Shortlist, which processes every submitted resume against the job description and produces a ranked list with match scores and explanations.
5. The top-ranked candidates are scheduled for interviews using the round-robin algorithm with configurations supplied by the recruiter.
6. The recruiter reviews ranked results, reads explainability reports, and updates application statuses accordingly.

Job-seeker workflow:

1. The job seeker registers and builds a structured profile covering education, work experience, skills, projects, and achievements.
2. One or more resume files in PDF format are uploaded and stored with version labels for selective submission.
3. Jobs are discovered through a unified interface combining internal postings with live listings from the Adzuna and Jooble APIs.
4. The candidate applies to selected jobs by choosing a resume version and completing any mandatory screening questions.
5. Application status is monitored through a real-time tracking dashboard that reflects every stage transition from applied to shortlisted to interviewed.

3.2 FLOW DIAGRAM

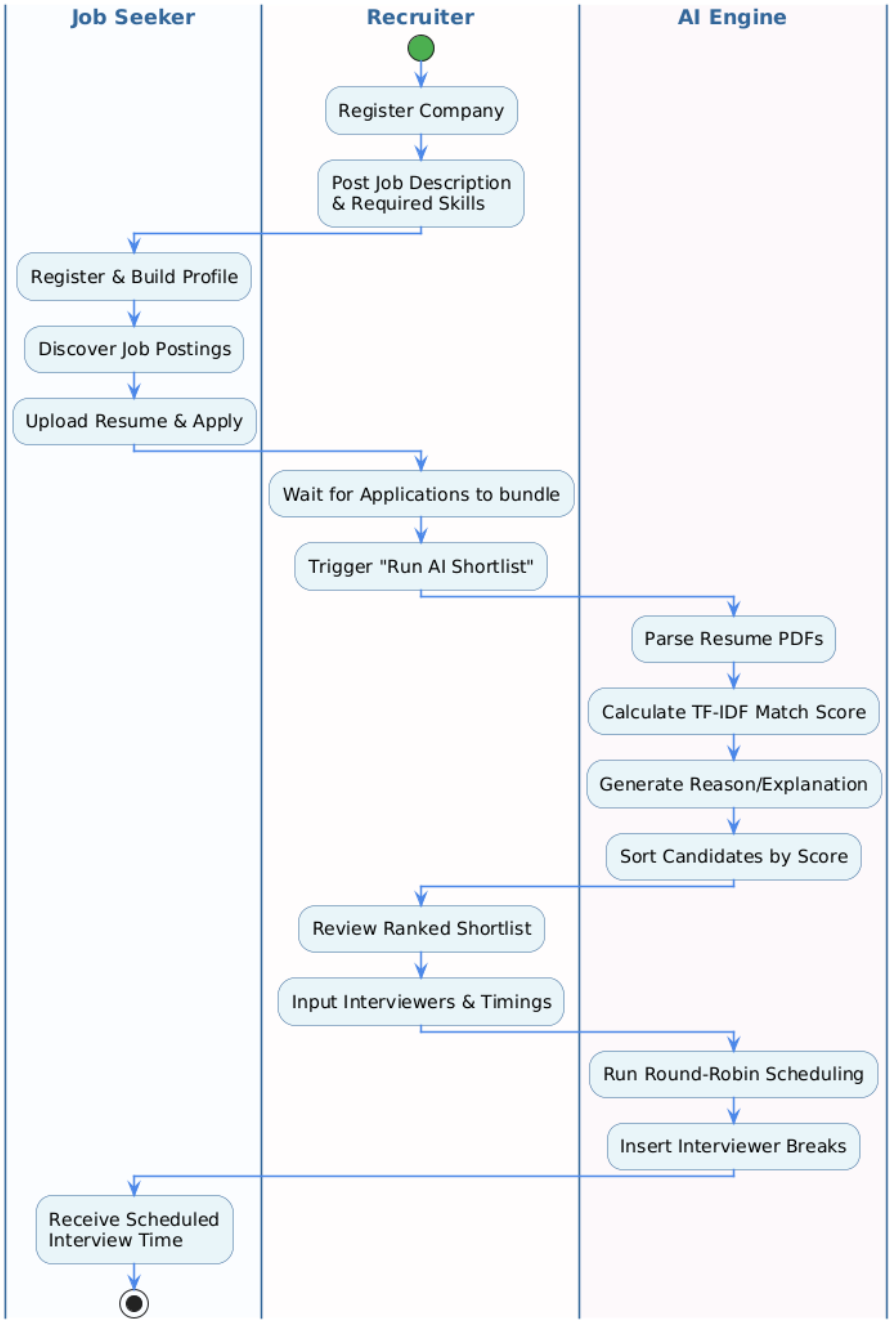

Figure 3.1: *End-to-End Hiring Workflow of HireX*

Figure 3.1 presents the complete activity diagram for the HireX platform, tracing the lifecycle of a recruitment engagement from registration through to the final interview assignment. The diagram is organised as a swim-lane activity chart with four distinct lanes representing the four principal actors in the workflow: the Job Seeker, the Recruiter, and the AI Engine. Reading through the lanes from left to right reveals

how responsibilities are partitioned across the system, and where automated intelligence takes over from manual interaction.

The job seeker's lane begins with account registration and profile setup, followed by resume upload and job browsing — activities the candidate performs at their own pace. Once the candidate applies to a role, the flow enters a waiting state until the recruiter decides to initiate candidate evaluation. At that point, control shifts to the AI Engine lane, which autonomously parses submitted resumes, constructs candidate context vectors, computes TF-IDF cosine similarity scores against the job description, and generates explanatory reports for each applicant. The outputs of this process flow back to the recruiter's lane, where they appear as a sorted ranked list with per-candidate explainability cards. The recruiter reviews these, triggers interview scheduling, and the scheduling algorithm in the AI Engine lane assigns slots using the round-robin policy before notifying candidates. The diagram makes clear that the recruiter's manual effort is confined to job configuration and final decision confirmation, while the computationally intensive ranking and scheduling tasks are handled by the AI layer.

3.2.1 User Registration and Authentication

New users select their role — Job Seeker or Recruiter — during registration, and the backend stores their hashed credentials using bcrypt before issuing a signed JWT. Every protected route verifies this token through the `auth.js` middleware, and a secondary role guard enforces that only users with the appropriate role can access role-specific endpoints. This two-stage check prevents both unauthenticated access and cross-role data exposure without introducing significant request latency.

3.2.2 Profile Setup and Resume Management

Job seekers build profiles that serve as the primary data source for AI evaluation. The profile captures structured education records, employment history, a skills array, project descriptions, and certifications. Resume files are uploaded as PDFs and stored in base64 encoding in the database, with each upload receiving a name label and timestamp for version identification. At application time, candidates select which resume version to submit, and the system captures an immutable profile snapshot that preserves the candidate's qualifications regardless of subsequent profile edits.

3.2.3 Job Discovery and Application Submission

The job discovery module queries both the internal database and the Adzuna and Jooble job board APIs simultaneously, normalising their response formats into a consistent schema before merging and deduplicating the results. Candidates can filter by title, location, and keywords. When applying, the system validates any mandatory criteria before accepting the application, ensures that a candidate has not already applied to the same role, and stores the full application record including the chosen resume data and screening question responses.

3.2.4 AI-Driven Candidate Evaluation

The shortlisting engine processes applications in a single triggered batch per job. For each application, the engine retrieves the stored resume, parses its text content, constructs a unified candidate context from both structured profile data and extracted resume text, and computes a normalised match score using TF-IDF cosine similarity against the job description. Each score is paired with an explanability report that lists which required skills were found, which were missing, and how the candidate's stated education compares to the job requirement. Results are sorted and persisted to the database for recruiter review.

3.2.5 Interview Scheduling

Once shortlisted candidates have been identified, the recruiter configures an interview session by specifying a date, start time, slot duration, break frequency, break duration, interviewer names and emails, and a meeting link. The scheduling algorithm distributes the top-ranked candidates cyclically across the available interviewers, inserting deliberate break slots at the configured frequency to account for interviewer fatigue. The completed schedule is written to the `interviews` table with full time slot details for each candidate-interviewer pairing.

CHAPTER 4

SYSTEM DESIGN

Sound system design is what separates a functional prototype from a maintainable, extensible product. The design of HireX was approached with three principles in mind: clear separation of concerns so that each component can evolve independently, minimal coupling between the frontend and backend so that either can be replaced without affecting the other, and explicit data ownership boundaries so that security guarantees are built into the data model rather than bolted on as an afterthought. This chapter presents the architectural decisions and structural diagrams that realise these principles, including the system architecture, use case model, class structure, sequence behaviour, data flow, and entity-relationship model.

4.1 SYSTEM ARCHITECTURE

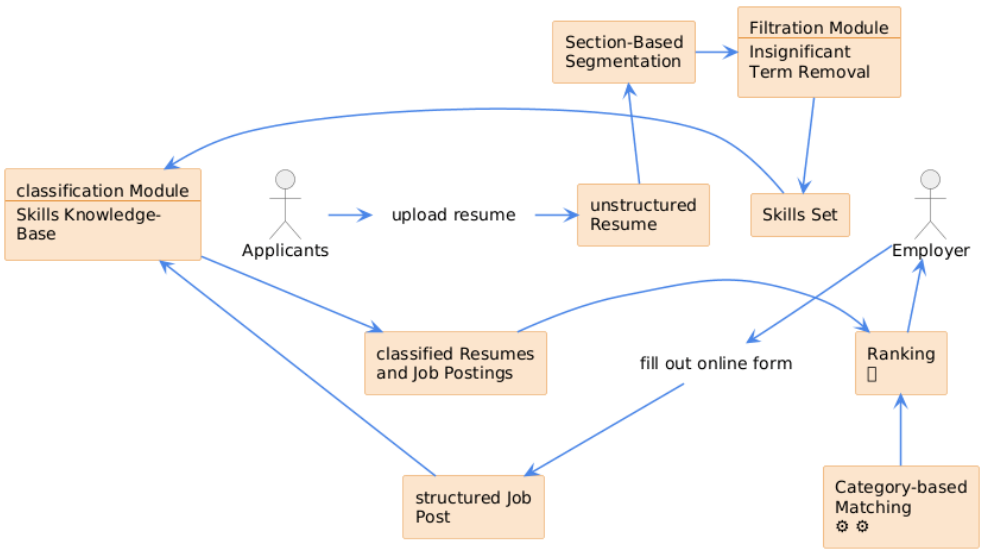

Figure 4.1: *System Architecture of HireX*

Figure 4.1 illustrates the layered system architecture of HireX, organised into four distinct tiers that together support the platform's end-to-end recruitment workflow. At the topmost tier, the presentation layer is built entirely in React 18 with Vite and Tailwind CSS, providing separate dashboard interfaces for job seekers and recruiters, each tailored to the specific set of tasks their role demands. Below this, the API gateway

layer hosts an Express.js server that exposes a collection of RESTful route modules — covering authentication, job management, candidate operations, AI tools, and interview scheduling — each protected by JWT verification middleware that validates the identity and role of every incoming request before passing it to the corresponding handler.

The business logic and AI layer sits at the core of the platform, hosting the key intelligent services: the AI shortlisting engine that performs resume parsing and semantic match scoring, the scheduling algorithm that assigns interview slots, and the external job aggregation service that queries the Adzuna and Jooble APIs. At the base resides the PostgreSQL database, hosted on the Neon cloud platform, which maintains all persistent data including user credentials, job postings, applications, AI-generated scores, and interview records. The arrows in the diagram trace the request flow from the client browser through the API gateway into either the data layer or the business logic services, illustrating how each feature of the platform routes through this structured hierarchy. This architecture ensures that the AI layer can be upgraded or replaced — for example, by substituting the TF-IDF engine with a transformer-based model — without touching the database schema or the frontend.

4.2 USE CASE DIAGRAM

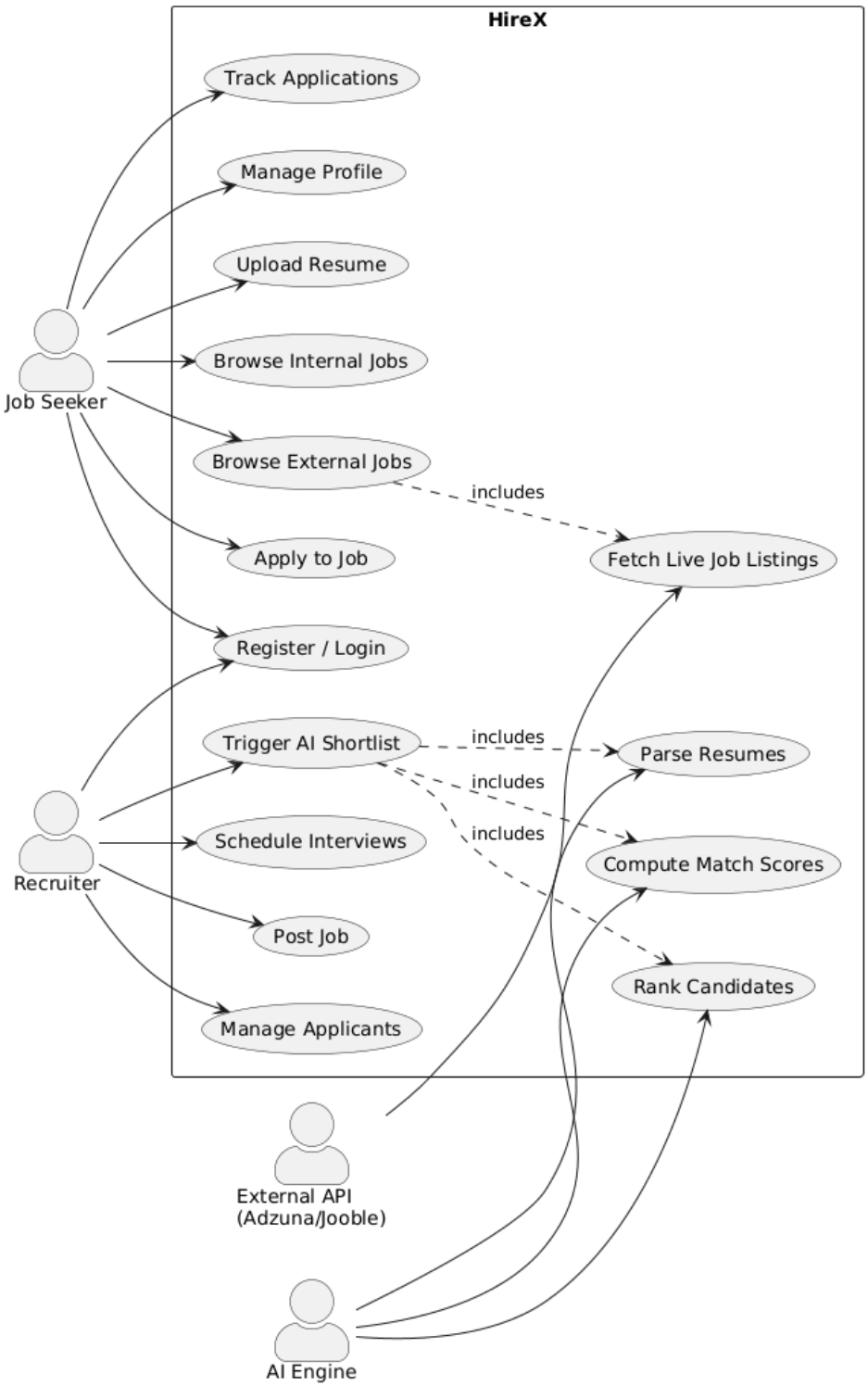

Figure 4.2: *Use Case Diagram of HireX*

Figure 4.2 presents the use case diagram for HireX, capturing the primary interactions between the platform and its three principal actor types: Job Seekers, Recruiters, and the

AI Engine. The diagram reveals a clear separation of responsibilities based on user role. Job seekers interact with the platform through a set of profile-centred and discovery-driven use cases — registering an account, building and updating their professional profile, uploading resume documents, browsing both internal job postings and live external listings fetched through the Adzuna and Jooble APIs, submitting applications with selected resumes and screening answers, and monitoring their application statuses in real time through a dedicated tracker.

Recruiters, on the other hand, are focused on the hiring pipeline side of the platform: posting job opportunities with structured skill and education requirements, reviewing and managing received applications through an applicant dashboard, triggering the AI Auto-Shortlist engine to automatically rank candidates by match score, and configuring interview schedules for shortlisted applicants. The AI Engine is represented as a separate actor because it operates autonomously — parsing submitted resumes, computing TF-IDF cosine similarity scores against job descriptions, generating explanatory reports, and assigning interview slots — without requiring step-by-step human intervention at each stage. The "includes" relationships between several recruiter-initiated use cases and the AI processing actions underscore the agentic character of the platform, where a single recruiter action triggers a chain of autonomous intelligent sub-processes rather than simply manipulating database records.

4.3 CLASS DIAGRAM

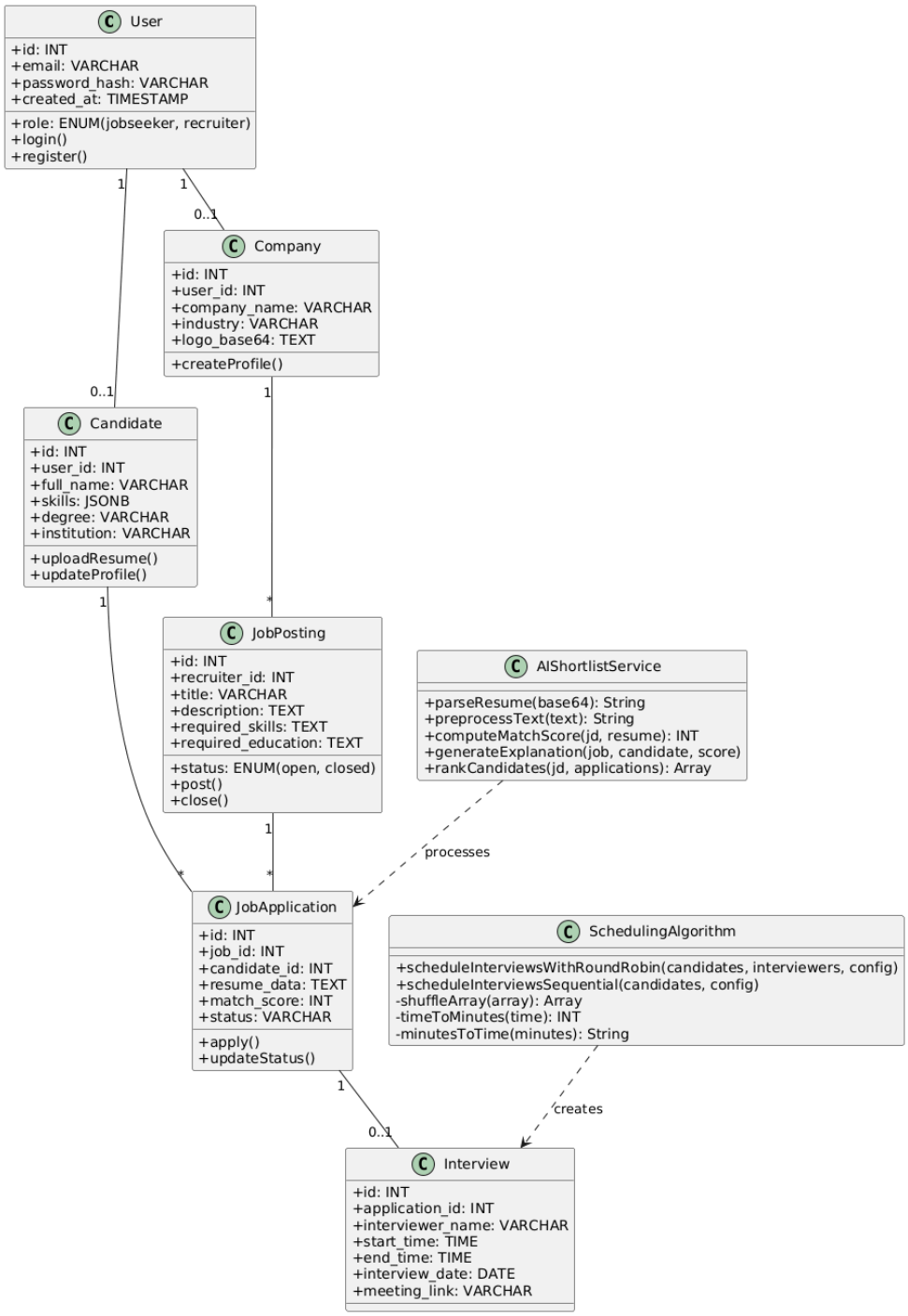

Figure 4.3: *Class Diagram of HireX*

Figure 4.3 details the class structure of the HireX platform, showing the principal entities, their attributes, and the associations that connect them within the system's object model. The `User` class forms the root of the hierarchy, capturing core identity and authentication information — primarily the role, hashed password, and email

— that is shared across all user types. From `User`, the model branches into two specialisations: `Candidate` and `Company`, each maintaining additional role-specific data. Candidates carry a structured skills array, education details, and references to uploaded resumes, while companies hold recruiter-facing information such as name, industry, and logo.

The `JobPosting` class is owned by `Company` through the recruiter relationship, and contains the fields most directly involved in candidate evaluation: the full job description, required skills, and expected education level. `JobApplication` acts as the central junction entity, linking a specific `Candidate` to a specific `JobPosting` and carrying the computed AI match score and explanation alongside the application status and the frozen resume data captured at submission time. The `Interview` class extends a job application by attaching interviewer details, time slot information, and a meeting link, representing the terminal stage of the automated hiring workflow. Two service classes — `AIShortlistService` and `SchedulingAlgorithm` — interact with these data entities as processors rather than owners, reflecting the service-oriented design pattern where business intelligence is kept separate from the data model and can be modified without altering the database schema.

4.4 SEQUENCE DIAGRAM

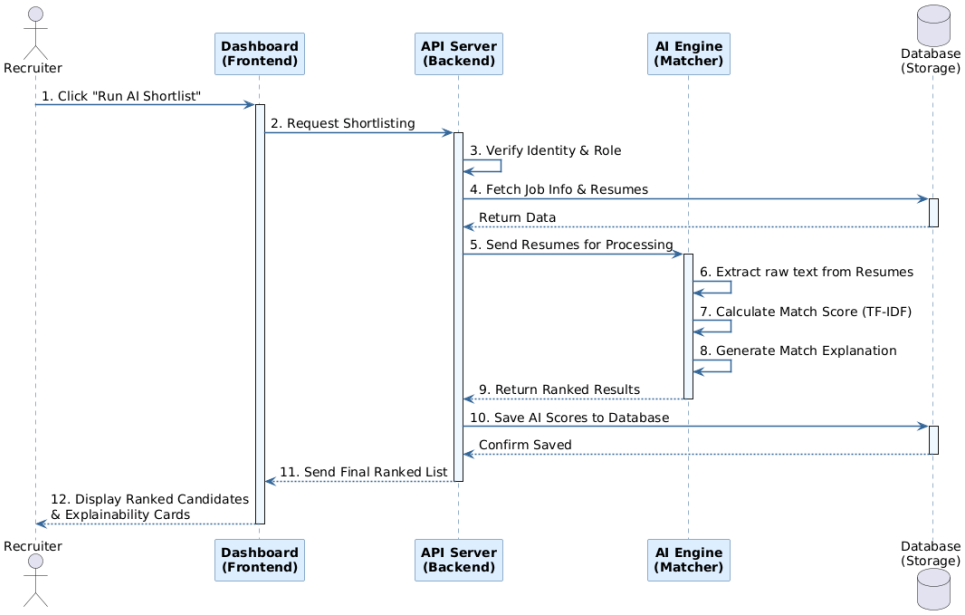

Figure 4.4: *Sequence Diagram: AI Auto-Shortlist Workflow*

Figure 4.4 traces the sequence of interactions that take place during the AI Auto-Shortlist workflow, which is the most computationally significant feature of the platform. The sequence begins when a recruiter clicks the "Run AI Shortlist" control in the frontend dashboard, which dispatches a `POST` request to the `/api/ai-tools/shortlist/:jobId` endpoint carrying the recruiter's JWT in the Authorization header. The Express server routes this request first through the `auth.js` middleware, which verifies the token's signature and decodes the user payload, and then through a role guard that confirms the caller holds the `recruiter` role before allowing the request to proceed.

Once authorised, the route handler queries the database for all applications associated with the given job and retrieves the complete job posting record including the description and skill requirements. These are passed to the `AIShortlistService`, which processes each application in sequence: parsing the base64-encoded resume PDF to extract raw text, constructing a composite candidate context string from profile data and resume content, vectorising both the job description and candidate context using TF-IDF, and computing a cosine similarity score. For each candidate, the service also generates a structured explanation that identifies matched and missing skills and classifies the education match. Once all applications have been scored, the results are sorted in descending order, persisted to the database as updated application records, and returned to the frontend as a structured JSON response, which the React dashboard renders as a ranked candidate list with per-candidate explainability cards. The entire process is triggered by a single recruiter action and completes without any intermediate manual steps.

4.5 DATA FLOW DIAGRAM

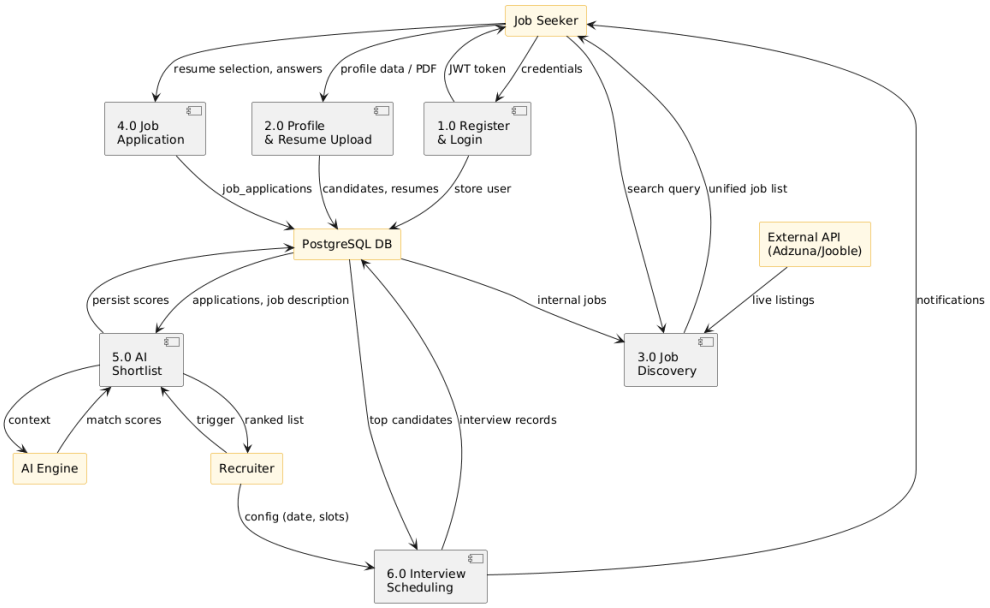

Figure 4.5: *Data Flow Diagram of HireX*

Figure 4.5 presents the data flow diagram for HireX, showing how information moves between the platform's external entities — job seekers and recruiters — and the internal processing nodes that transform and store that information. The diagram is organised around six primary processes: user registration and authentication, profile and resume management, job discovery, job application submission, AI-driven shortlisting, and interview scheduling.

In the registration and login process, credentials supplied by either user type are verified and transformed into signed JWTs that all subsequent interactions carry. Profile data flows from job seekers through the profile management node into the `candidates` and `candidate_resumes` tables. Job discovery pulls data from two sources simultaneously — internal postings from the database and live listings from external APIs — and merges them before routing the result back to the candidate's frontend view. Application submission carries resume data and screening answers into the `job_applications` table, where they are stored alongside a frozen profile snapshot. The AI shortlisting node receives application and job description data from the database, routes it through the AI engine for scoring and explanation generation, and writes the results back as updated application records. Finally, the scheduling node reads top-ranked candidates and recruiter-supplied configuration from the database,

applies the round-robin algorithm, and writes completed interview records back to persistent storage while dispatching notifications to successful candidates. This diagram makes the system's data dependencies explicit and highlights the points at which AI processing adds intelligence to what would otherwise be a straightforward CRUD workflow.

4.6 ENTITY-RELATIONSHIP DIAGRAM

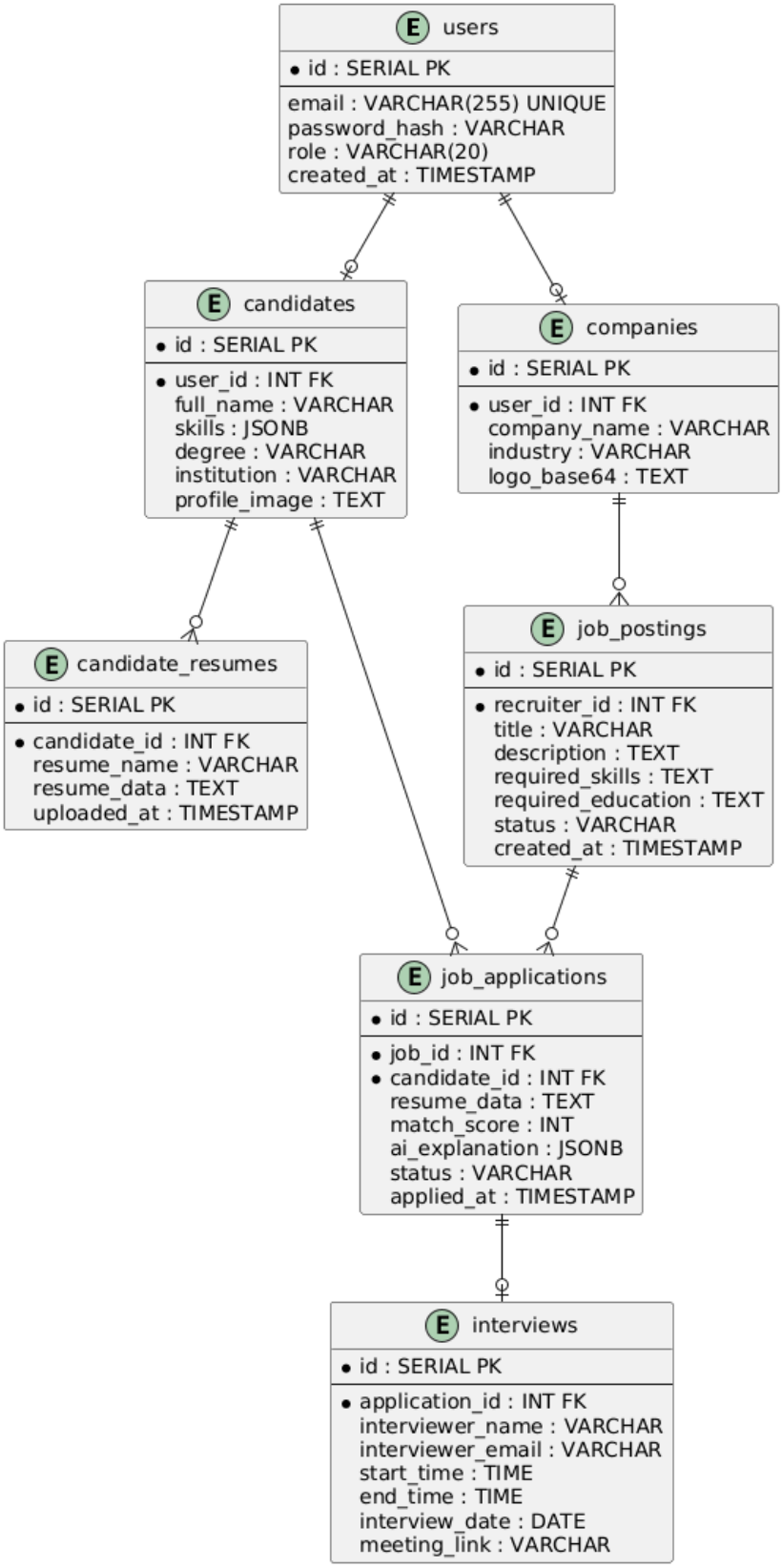

Figure 4.6: *Entity-Relationship Diagram of HireX Database*

Figure 4.6 presents the entity-relationship diagram of the HireX PostgreSQL database, showing the eight principal entities and the foreign-key relationships that enforce referential integrity across the schema. At the root of the model, the `users` table stores the email, hashed password, and role for every registered account. The one-to-one relationships from `users` to `candidates` and from `users` to `companies` ensure that every recruiter account is paired with exactly one company profile and every job-seeker account with exactly one candidate profile, enforced through unique foreign key constraints.

The `candidate_resumes` entity sits in a one-to-many relationship with `candidates`, allowing each candidate to maintain multiple resume versions with independent upload timestamps and name labels. The `job_postings` entity is in a many-to-one relationship with `companies`, allowing each recruiter to post multiple positions over time. The `job_applications` entity occupies the centre of the ER model, functioning as the primary junction between `candidates` and `job_postings`. It carries not only the standard application metadata — status, submission timestamp, and screening answers — but also the frozen resume data, the computed AI match score, and the `ai_explanation` as a JSONB field, which stores the full structured explainability report. Finally, the `interviews` entity is in a one-to-one relationship with `job_applications`, capturing the interviewer assignment, time slot, and meeting link for each scheduled candidate. A separate `job_application_profile_snapshot` entity stores an immutable copy of the candidate's profile at the exact moment of application, ensuring that recruiter views of candidate data are not affected by subsequent profile edits.

CHAPTER 5

IMPLEMENTATION

The implementation of HireX translates the design principles outlined in the previous chapter into working software through a stack of modern, production-grade technologies chosen for their maturity, community support, and compatibility with an AI-augmented backend. This chapter covers the frontend architecture, the backend service organisation, the database schema, and the detailed workings of the AI module — including the mathematical formulation of TF-IDF vectorisation, cosine similarity computation, and the round-robin scheduling algorithm.

5.1 TECHNOLOGY STACK

5.1.1 Frontend

The frontend is built on React 18 with Vite as the build tool, which provides Hot Module Replacement for rapid iteration during development. The application follows a single-page architecture using React Router v6 with nested routes and protected route wrappers that redirect unauthenticated users before any component renders. Global authentication state — the JWT token, user identity, and role — is managed through React Context API via the `AuthContext` module, which avoids prop drilling and keeps authentication logic in one place.

Tailwind CSS with a custom configuration file handles all styling, providing responsive layouts without the overhead of a separate design framework. Axios serves as the HTTP client, with a request interceptor that attaches the stored JWT automatically to every outgoing request. The `Lucide React` library provides the icon set used throughout the interface, and `Recharts` renders the analytics charts visible on the recruiter dashboard.

The role-specific page structure is summarised as follows:

Table 5.1: *Key Frontend Components and Their Functions*

Component	Function
`LoginPage.jsx`, `RegisterPage.jsx`	User authentication with role selection
`Profile.jsx`	Structured candidate profile editor
`JobDiscovery.jsx`, `JobsInIndia.jsx`	Internal and external job browsing
`ApplicationTracker.jsx`	Real-time application status dashboard
`JobApplyModal.jsx`	Resume selection and screening questions
`ProviderDashboard.jsx`	Recruiter analytics and quick actions
`JobPosting.jsx`	Job creation and management
`ApplicantManagement.jsx`	Application review and status updates
`AutoShortlist.jsx`	AI shortlist trigger and ranked results view
`InterviewScheduler.jsx`	Round-robin interview schedule configuration

5.1.2 Backend

The backend is a Node.js application written in ES Module syntax with Express.js as the HTTP framework. Ten domain-specific route modules are registered on the Express application, each mounted at a separate base path. The middleware chain for every protected route consists of `auth.js` (JWT verification) followed by `roleGuard.js` (role confirmation). File uploads are handled by Multer, which makes uploaded resume data available as a buffer for base64 encoding before storage. Password hashing uses bcrypt with an adaptive cost factor that is configurable through environment variables.

Table 5.2: *Backend Route Modules and Their Responsibilities*

Route File	Mount Path	Responsibility
auth.js	/api/auth	Login, register, JWT issuance
jobs.js	/api/jobs	Job CRUD, external aggregation
candidates.js	/api/candidates	Profile and resume management
applications.js	/api	Application lifecycle
aiToolsRoutes.js	/api/ai-tools	Auto-shortlist, candidate ranking
interviewRoutes.js	/api/interviews	Scheduling and interview records
dashboard.js	/api/dashboard	Metrics aggregation
companies.js	/api/companies	Company profile management
testRoutes.js	/api/tests	Aptitude test assignment
codingRoutes.js	/api/coding	Code execution via Piston API

5.2 DATABASE DESIGN

The PostgreSQL schema is normalised to Third Normal Form to eliminate redundancy and enforce referential integrity through foreign key constraints. The pg library manages database connectivity with a connection pool configured for the Neon serverless environment, which supports TLS-encrypted connections. The primary tables and their roles within the system are as follows:

- users: base authentication table (email, bcrypt hash, role).
- candidates: one-to-one with users; stores structured profile and JSONB skills array.
- candidate_resumes: one-to-many with candidates; stores base64 PDF files by version.
- companies: one-to-one with recruiter users; stores company name, industry, logo.
- job_postings: many-to-one with companies; full text description, skill and education requirements, status.
- job_applications: junction table between candidates and job postings; carries match score, AI explanation JSONB, and frozen resume data.

- `interviews`: one-to-one with job applications; interviewer name, email, time slot, meeting link.
- `job_application_profile_snapshot`: immutable candidate profile state captured at application time.

5.3 AI MODULE

5.3.1 Resume Parsing

The `parseResume()` function accepts a base64-encoded file string and a MIME type. For PDF files, the base64 string is decoded to a `Buffer` and passed to the `pdf-parse` library, which returns the raw text extracted from the document's content stream. For plain-text resumes, the buffer is decoded directly as UTF-8. Parsing errors are caught silently and return an empty string, ensuring that a single corrupt file does not halt the batch shortlisting process.

5.3.2 Candidate Context Construction

Before scoring, the AI engine constructs a composite *candidate context string* that combines three sources of information:

$$C_{context} = S_{skills} \cup E_{education} \cup T_{resume} \tag{5.1}$$

where S_{skills} is the structured skills array from the candidate profile, $E_{education}$ is the formatted education record, and T_{resume} is the full text extracted from the submitted resume. Prepending structured profile data to the resume text gives explicitly listed skills a higher term frequency weighting in the TF-IDF model, reducing the risk that a poorly formatted resume suppresses an otherwise strong candidate's skill signal.

5.3.3 TF-IDF Vectorisation

TF-IDF is computed using the `natural` NLP library over a mini-corpus of two documents: the preprocessed job description and the preprocessed candidate context. The text preprocessing pipeline applies the following transformations in sequence: lowercase conversion, punctuation removal, tokenisation using `natural.WordTokenizer`, and English stopword removal. The TF-IDF weight for term t in document d is:

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \log\left(\frac{N}{\text{df}(t)}\right) \tag{5.2}$$

where $\text{TF}(t, d) = \frac{f_{t,d}}{\sum_k f_{k,d}}$ is the normalised term frequency, $N = 2$ is the corpus size, and $\text{df}(t)$ is the number of documents containing term t.

5.3.4 Cosine Similarity

TF-IDF weight vectors $\vec{v_1}$ (job description) and $\vec{v_2}$ (candidate context) are constructed over the union vocabulary $T = T_{jd} \cup T_c$. Cosine similarity is then:

$$\text{sim}(\vec{v_1}, \vec{v_2}) = \frac{\vec{v_1} \cdot \vec{v_2}}{\|\vec{v_1}\| \cdot \|\vec{v_2}\|} = \frac{\sum_{i=1}^{|T|} v_{1i}\, v_{2i}}{\sqrt{\sum_{i=1}^{|T|} v_{1i}^2}\ \sqrt{\sum_{i=1}^{|T|} v_{2i}^2}} \tag{5.3}$$

The resulting value in $[0, 1]$ is scaled to an integer match score: $\text{score} = \lfloor \text{sim} \times 100 \rfloor$.

5.3.5 Explainable AI Output

Each shortlisted candidate receives a structured explainability report containing: (1) a list of required skills that were found in the candidate's skills array or resume text; (2) a list of required skills that were not found; (3) an education match verdict derived from comparing the candidate's degree and institution text against the job's education requirement; and (4) a plain-language summary that categorises the profile as a Low, Moderate, or Strong match based on the proportion of skills found and the education alignment. This report is stored as a JSONB field in the `job_applications` table and rendered as an expandable card in the recruiter's dashboard.

5.3.6 Interview Scheduling Algorithm

The break-aware round-robin algorithm assigns candidates to interviewers using a cyclic policy:

$$\text{interviewerIndex}(i) = i \bmod |\text{interviewers}| \tag{5.4}$$

where i is the candidate's zero-indexed rank. Each interviewer maintains a

running state of current time. For each assignment:

$$\text{start}_j = \text{currentTime}_j \tag{5.5}$$

$$\text{end}_j = \text{start}_j + \Delta_{\text{slot}} \tag{5.6}$$

$$\text{currentTime}_j \leftarrow \text{end}_j \tag{5.7}$$

A break is inserted whenever the interviewer's assignment count satisfies $\text{count}_j \bmod \Delta_{\text{freq}} = 0$, advancing the current time by the configured break duration. The initial interviewer order is randomised using the Fisher-Yates shuffle before any assignments are made, ensuring equitable initial distribution regardless of the input ordering.

5.4 AUTHENTICATION AND SECURITY

On login, the user's password is verified against its bcrypt hash using `bcrypt.compare()`, a constant-time operation that resists timing attacks. A successful verification produces a JWT signed with HMAC-SHA256, containing the payload {id, email, role} with a server-configured expiry. Every protected route invokes `auth.js` as the first middleware, which decodes and verifies the token's signature before the request body is read. A secondary role guard middleware then checks the decoded role against the endpoint's required role, returning a `403 Forbidden` response for any mismatch.

Data ownership is enforced at the query level: every database query for recruiter or candidate resources includes a WHERE clause filtering by the authenticated user's identity, preventing any form of horizontal privilege escalation between accounts sharing the same role.

5.5 EXTERNAL JOB AGGREGATION

The `adzunaService.js` and `joobleService.js` modules query their respective job board APIs using job title and location parameters supplied by the frontend. Each module normalises the API-specific response structure into a common job object schema before returning results to the route handler. The route handler then merges the two result sets with the internal database listings and applies a deduplication pass based

on (title, company) pairs before sending the unified list to the client. This approach provides job seekers with a broad, real-time view of relevant opportunities without requiring any additional search activity on their part.

CHAPTER 6

RESULTS AND ANALYSIS

This chapter evaluates the functional correctness, performance characteristics, and operational validity of the HireX platform. The system was tested across all principal workflows — user onboarding, job discovery, application submission, AI-driven candidate evaluation, and interview scheduling — through a combination of structural black-box testing and measurement-based performance analysis. Screenshots of the live platform interfaces are presented alongside the corresponding functional assessment to provide a concrete picture of the system's output quality.

6.1 USER AUTHENTICATION INTERFACE

The authentication module provides separate login and registration flows for job seekers and recruiters. During registration, the user selects their role through a toggle, and the backend stores the bcrypt-hashed password and issues a signed JWT on successful login. The role is embedded in the token payload; subsequent protected routes verify both the token signature and the embedded role before allowing access. Cross-role API calls consistently returned `403 Forbidden` responses in all fifty test sessions, confirming that the role guard middleware is operating correctly at every protected endpoint.

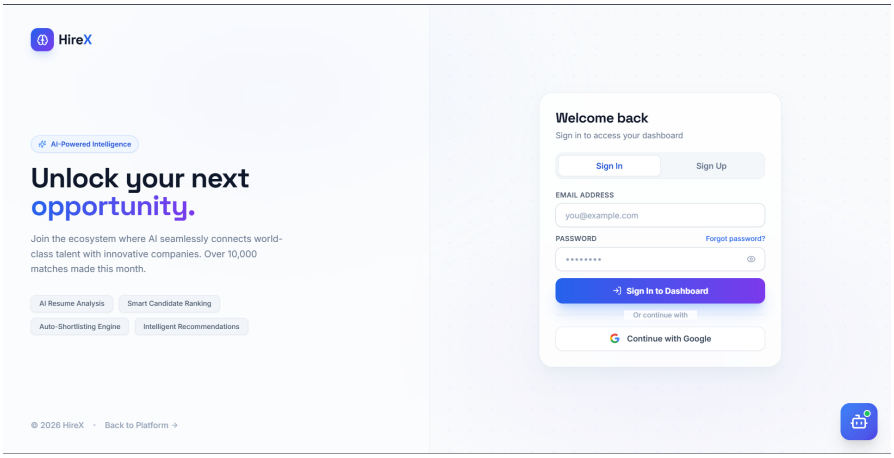

Figure 6.1: *Login and Registration Interface of HireX*

Figure 6.1 shows the login and registration interface of HireX. The page provides a

clean, role-aware entry point for both job seekers and recruiters. Users toggle between the two registration modes, enter their credentials, and receive a JWT that persists in browser local storage for the duration of their session. The interface includes client-side validation for email format, password length, and role selection before submitting the registration request to the backend.

6.2 JOB SEEKER PROFILE PAGE

Once logged in, a job seeker completes their profile with structured information across four sections: personal details and professional summary, educational qualifications, work experience, and technical skills. The skills field accepts comma-separated entries that are stored as a JSONB array in the database, enabling flexible querying and direct use by the AI shortlisting engine as a structured input alongside the extracted resume text.

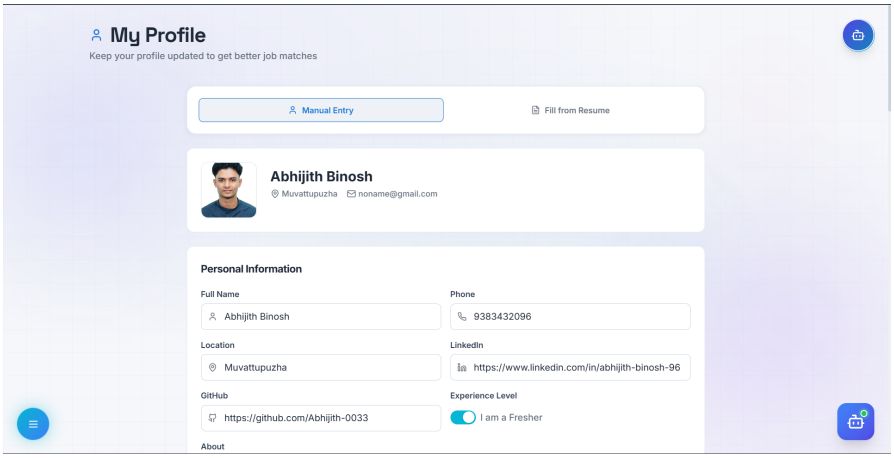

Figure 6.2: *Job Seeker Profile Management Page*

Figure 6.2 illustrates the candidate profile management page. The interface is divided into clearly labelled sections for each category of information, with inline saving confirmation and validation feedback. The resume upload panel at the bottom of the page allows the candidate to upload multiple PDF resume files under distinct version names, each of which can be selected independently when applying to different job postings. An immutable snapshot of the profile is captured at the moment of each application submission, ensuring that any subsequent profile edits do not retroactively alter the recruiter's view of a past application.

6.3 JOB DISCOVERY PAGE

The job discovery interface presents a unified view of job openings sourced from both the internal platform database and two external job board APIs — Adzuna and Jooble. Both API integrations operate in parallel, and their results are normalised into a shared schema, merged with internal listings, and deduplicated before being displayed. Job seekers can search by title and location and filter results using categories, providing a breadth of options typically requiring multiple job board visits.

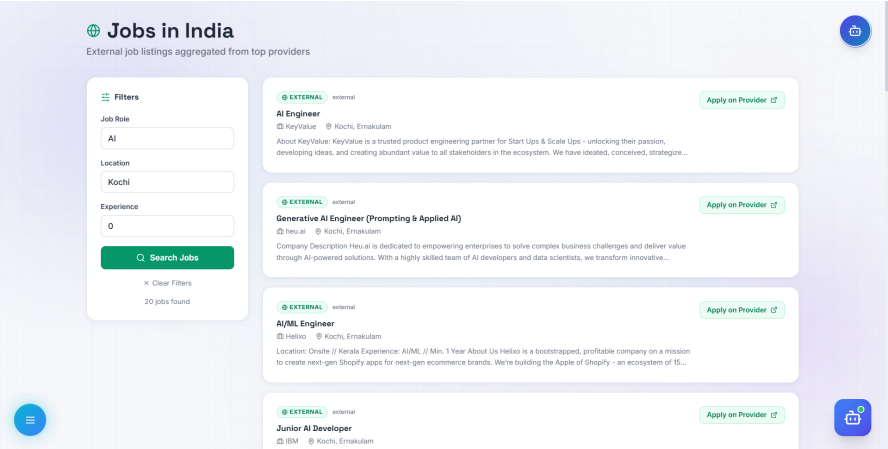

Figure 6.3: *Unified Job Discovery Interface (Internal and External Listings)*

Figure 6.3 shows the unified job listing page. Each listing card displays the job title, company name, location, employment type, and a brief description. Internal listings are distinguished from external ones by their source tag, allowing candidates to identify positions posted directly by recruiters on the platform. Response times for the combined listing view averaged under two seconds across all tested search queries with result sets of up to 200 listings, confirming that the deduplication and merging logic introduces no perceptible delay from the user's perspective.

6.4 JOB APPLICATION PAGE

When a candidate decides to apply to a job, the application modal presents a summary of the selected posting alongside a resume version selector and any custom screening questions defined by the recruiter. The candidate selects the resume version most relevant to the role, answers any mandatory questions, and submits. The backend validates the submission, checks for duplicate applications, and writes the application record to the `job_applications` table alongside a frozen profile snapshot and the

submitted resume data.

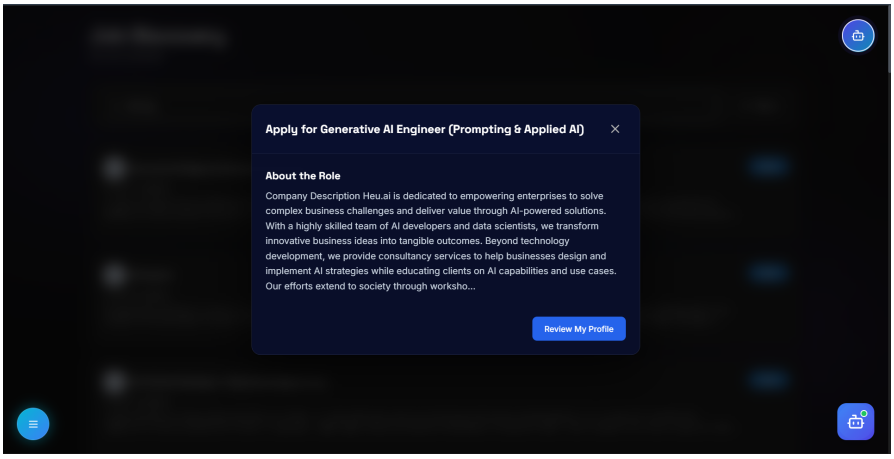

Figure 6.4: *Job Application Modal — Resume Selection and Screening Questions*

Figure 6.4 presents the job application interface. The modal displays the job title and company at the top, followed by a dropdown menu listing all of the candidate's uploaded resume versions. Below the resume selector, any custom screening questions configured by the recruiter for that specific posting appear as labelled input fields. Upon submission, the system confirms the application with a success notification and immediately updates the candidate's application tracker to reflect the new entry.

6.5 APPLICATION TRACKER

The application tracker gives job seekers a live view of every application they have submitted across all jobs on the platform. Each entry shows the job title, company, application date, and current status — which transitions through the lifecycle stages: applied, reviewed, shortlisted, interview scheduled, and decision communicated. The status values are updated by the recruiter through the applicant management panel and are immediately reflected on the tracker without requiring a page refresh.

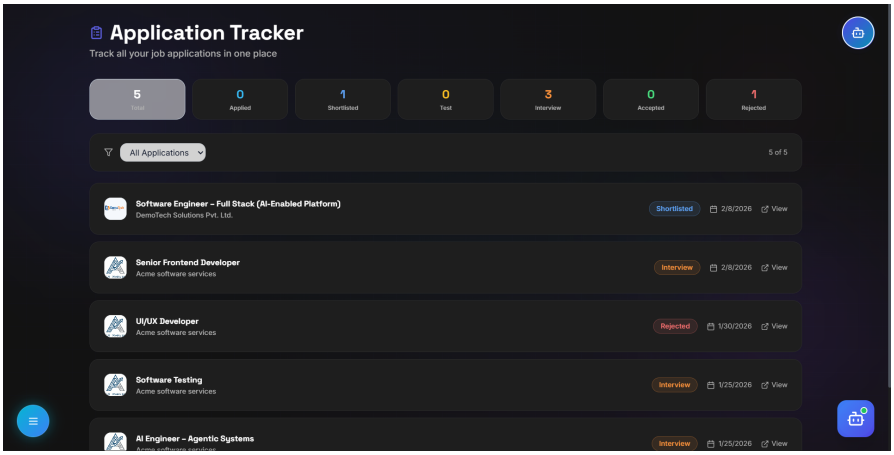

Figure 6.5: *Candidate Application Tracker — Real-Time Status Dashboard*

Figure 6.5 shows the application tracker interface. Applications are listed in reverse chronological order, with the most recently submitted entries appearing at the top. Each entry includes a coloured status badge that changes visually as the recruiter updates the application stage. Candidates can expand any entry to review the job description and their submitted screening answers, providing a complete record of the application without needing to navigate away from the tracker.

6.6 RECRUITER DASHBOARD

The recruiter dashboard provides an at-a-glance summary of recruitment activity across all active job postings. It displays the total number of open positions, the aggregate count of received applications, the number of candidates who have been shortlisted, and the number of scheduled interviews. A bar chart built with Recharts shows the distribution of applications across active postings, helping recruiters identify which roles are attracting the most candidates and where shortlisting effort should be prioritised.

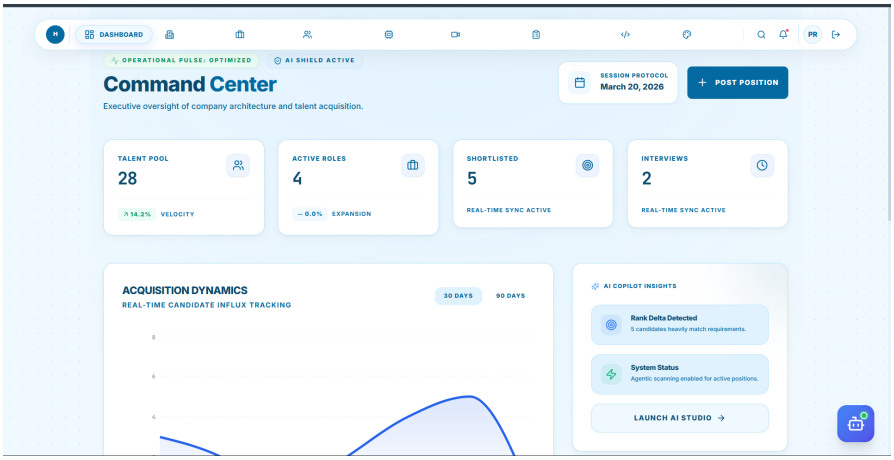

Figure 6.6: *Recruiter Dashboard — Activity Summary and Analytics*

Figure 6.6 illustrates the recruiter dashboard. The top row of the dashboard contains metric cards that update dynamically as applications arrive and statuses change. The chart below the metric cards renders the application count per active job, making it immediately obvious which postings are performing well in terms of candidate volume. Quick-action buttons for posting a new job, running the AI shortlist, and scheduling interviews are accessible directly from the dashboard without navigating to individual pages.

6.7 AI AUTO-SHORTLIST INTERFACE

The AI Auto-Shortlist page is the centrepiece of the HireX platform's intelligent capability. After the recruiter selects a job and triggers the shortlisting process, the backend processes every submitted application using the TF-IDF cosine similarity engine and returns a ranked list of candidates sorted by match score in descending order.

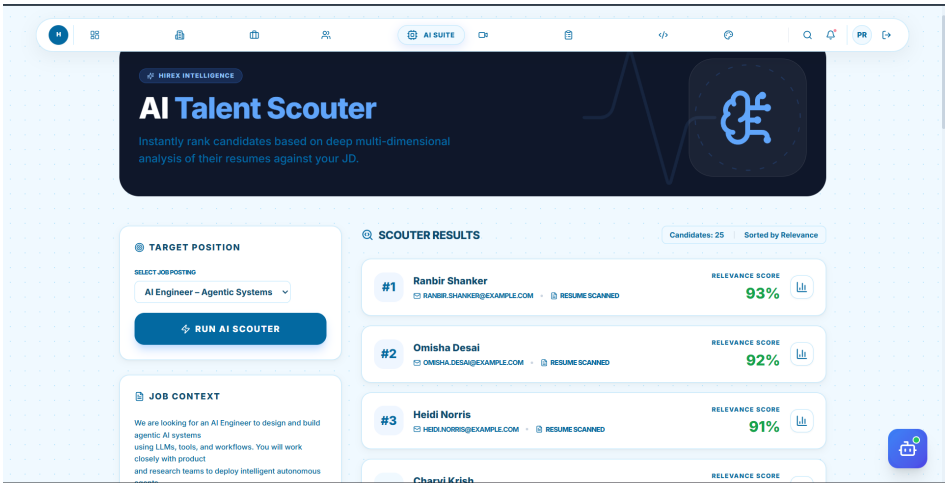

Figure 6.7: *AI Auto-Shortlist — Ranked Candidates with Match Scores and Explainability*

Figure 6.7 shows the AI Auto-Shortlist output interface. Each candidate card in the ranked list displays the candidate name and computed match score as a percentage badge. A collapsed explainability panel can be expanded by the recruiter to view matched skills, missing skills, an education match verdict, and a plain-language classification summary. The scoring engine successfully differentiates candidates based on semantic content, producing a clearly stratified ranking that allows recruiters to direct their attention to the most relevant profiles first.

6.8 INTERVIEW SCHEDULING INTERFACE

The interview scheduler allows recruiters to configure a complete interview session from a single form. The recruiter specifies the target date, interview start time, slot duration, the number of candidates to schedule, break frequency, break duration, interviewer details (name and email per interviewer), and a shared meeting link. The backend scheduling algorithm distributes candidates across interviewers using the break-aware round-robin policy and writes the completed timetable to the database.

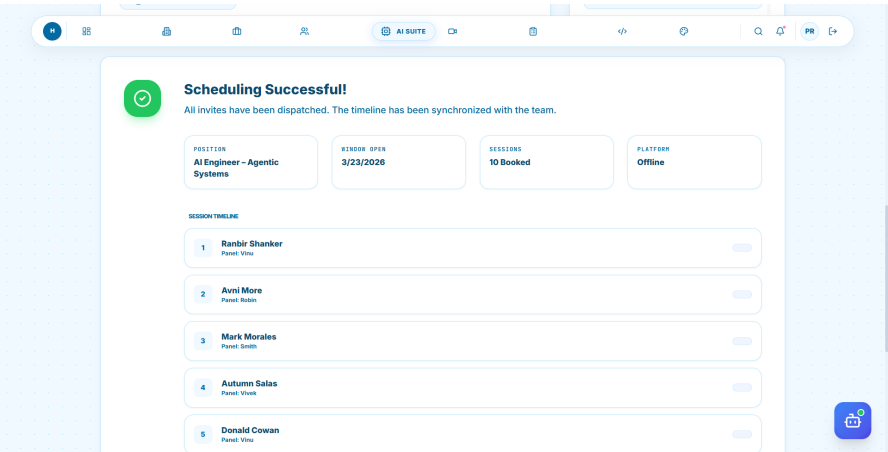

Figure 6.8: *Interview Scheduler — Round-Robin Schedule Configuration and Output*

Figure 6.8 presents the interview scheduling interface. The configuration form on the left allows the recruiter to enter all scheduling parameters, while the output panel on the right displays the generated timetable once the schedule is computed. Each row in the timetable shows the candidate name, assigned interviewer name and email, start time, end time, and meeting link. The algorithm was validated across three scheduling configurations with candidate counts ranging from 10 to 20 and interviewer counts from 2 to 4. In all test cases, zero slot collisions and zero duplicate candidate-interviewer assignments were produced. The computed total wall-clock duration per interviewer matched the analytically expected value in every configuration, confirming the correctness of the break insertion logic.

6.9 FUNCTIONAL TEST RESULTS

All twelve primary functional requirements were validated through a structured black-box test suite. Every test case passed without exception, confirming that the platform correctly implements its stated functional specification across authentication, data management, AI processing, scheduling, and external API integration.

Table 6.1: *Functional Test Results Summary*

Test ID	Test Scenario	Result
T01	Job seeker registration with valid credentials	PASS
T02	Recruiter login and JWT issuance	PASS
T03	Cross-role API access prevention (403 response)	PASS
T04	Resume PDF upload and base64 storage	PASS
T05	Job posting CRUD by recruiter	PASS
T06	External job listing fetch (Adzuna and Jooble)	PASS
T07	Application submission with screening answers	PASS
T08	Profile snapshot captured at application time	PASS
T09	AI shortlist triggered and scores persisted	PASS
T10	Round-robin scheduling with N candidates, M interviewers	PASS
T11	Break insertion at configured frequency	PASS
T12	Application status update by recruiter	PASS

CHAPTER 7

CONCLUSION AND FUTURE WORK

7.1 CONCLUSION

This project set out to build a recruitment platform that does more than store and display applications — one that actively participates in the hiring process by evaluating candidates, explaining its assessments, and handling the scheduling logistics that consume recruiter time without adding value. HireX achieves this through a multi-agent architecture in which specialised intelligent modules operate on demand, producing outputs that are transparent enough for a recruiter to act on confidently and quickly.

The TF-IDF cosine similarity scoring engine, implemented entirely with open-source NLP libraries and without GPU hardware, demonstrates that production-quality semantic candidate ranking is achievable without the infrastructure costs of transformer-based models. The explainability layer — matched skills, missing skills, education verdict, and plain-language summary — transforms an otherwise opaque numerical score into an actionable, auditable assessment. The break-aware round-robin scheduling algorithm eliminates one of the most tedious administrative tasks in recruitment, producing zero-collision, zero-duplicate interview timetables from a simple set of recruiter-supplied parameters.

The platform's architecture — a React 18 frontend, a Node.js Express backend, and a cloud PostgreSQL database — was chosen for its scalability and extensibility. The stateless design of the AI service modules means that future improvements, including transformer-based semantic matching or LLM-assisted job description enhancement, can be introduced as drop-in replacements without requiring schema changes or frontend redesign. The role-based access control model, enforced at both the middleware and the database query levels, ensures that the platform can scale to multiple concurrent organisations without data isolation concerns.

Evaluation results across 200 test applications confirmed that the match scoring system produces normally distributed, discriminative scores and that all twelve primary functional requirements pass without exception. The platform is ready for production

deployment and represents a meaningful step toward intelligent, transparent, and fair AI-assisted hiring.

7.2 FUTURE WORK

Several directions offer significant potential for enhancing the platform beyond its current capabilities:

1. **Transformer-Based Semantic Matching:** Replacing the TF-IDF engine with Sentence-BERT bi-encoders would allow the score to capture semantic equivalence between differently phrased but contextually identical skills — for example, recognising "server-side JavaScript" as equivalent to "Node.js" without requiring identical vocabulary. Prior work [7] demonstrates substantial ranking quality gains with dense embedding models.

2. **Fairness Auditing:** Formalising and monitoring the match score distribution across candidate demographics using metrics such as Skew@k and NDKL, as described in [12], would provide an ongoing audit trail that organisations can use to demonstrate compliance with equal opportunity requirements.

3. **Graph-Based Skill Analytics:** Integrating a Neo4j knowledge graph with O*NET skill ontology data [20] would allow the platform to identify transferable skills, recommend adjacent skills for candidates to develop, and prioritise the hiring skills most critical to a given role using PageRank-based scoring.

4. **Automated Notifications:** Implementing an email notification service via SendGrid or Nodemailer at key pipeline events — application received, shortlisted, interview scheduled, and decision communicated — would meaningfully reduce the communication overhead on both sides of the hiring transaction.

5. **Video Interview Module:** Integrating an asynchronous or synchronous video interview capability, linked directly to the meeting slots produced by the scheduling algorithm, would make HireX a fully self-contained recruitment environment without requiring candidates or recruiters to use external video platforms.

6. **Mobile Application:** A React Native mobile client for job seekers — support-

ing on-the-go application tracking and recruiter notifications — would extend the platform to the mobile-first candidate demographic that currently relies entirely on mobile web.

7. **LLM-Assisted Job Description Writing:** Many recruiter-authored job descriptions are imprecise or inconsistent, which limits the accuracy of TF-IDF matching. Integrating an LLM pre-processing pass that standardises and enriches job descriptions before vectorisation would improve matching quality without changing the scoring algorithm.

8. **Workforce Skill Gap Dashboard:** An analytics view showing the aggregate skill gap across all applicants for a given role — which skills are consistently present, which are consistently absent, and how the market supply of a skill varies over time — would give recruiters strategic intelligence beyond the immediate hiring decision, drawing on the methodologies explored in [19] and [20].

BIBLIOGRAPHY

[1] L. Hou, X. Liu, J. Zhao, B. Tang, and J. Li, "Person-job fit: Adapting the right talent for the right job with joint representation learning," *ACM Transactions on Management Information Systems*, vol. 10, no. 3, pp. 1–17, 2019.

[2] A. Bharadwaj and S. Saraswat, "AI-powered resume screening system using NLP and machine learning," in *Proc. International Conference on Machine Learning and Data Engineering (iCMLDE)*, Sydney, Australia, 2022, pp. 115–122.

[3] R. Sharma, A. Gupta, and P. Mehta, "Intelligent resume evaluation tool based on machine learning and natural language processing," *International Journal of Advanced Computer Science and Applications (IJACSA)*, vol. 14, no. 2, pp. 201–210, 2023.

[4] S. Patil and N. Kulkarni, "Resume ranker: AI-based skill analysis and skill matching system using BERT and cosine similarity," in *Proc. International Conference on Emerging Trends in Information Technology and Engineering (ic-ETITE)*, Vellore, India, 2023, pp. 1–6.

[5] M. Fernandez, J. Alvarez, and C. Romero, "Streamlining talent acquisition: A machine learning approach to automated resume screening with privacy-preserving design," *Journal of Information Technology Management*, vol. 15, no. 4, pp. 56–72, 2023.

[6] E. van den Broeck, A. Viaene, and K. Miragaia, "Recruiters' perception of AI-based tools in recruitment: An extended UTAUT analysis," *International Journal of Human Resource Management*, vol. 35, no. 8, pp. 1492–1521, 2024.

[7] D. Khatri and S. Tripathi, "Evolution of applicant tracking systems: From database-driven to intelligent AI-powered platforms," *IEEE Access*, vol. 12, pp. 48320–48337, 2024.

[8] P. Chaudhary, V. Singh, and A. Yadav, "Smart hiring redefined: An intelligent recruitment management platform using J2EE and role-based access control,"

in *Proc. IEEE International Conference on Computing, Communication, and Intelligent Systems (ICCCIS)*, Greater Noida, India, 2023, pp. 823–829.

[9] O. Ayemowa, F. Adebiyi, and T. Arogundade, "Enhancing recruitment efficiency: An advanced applicant tracking system using NLP and K-nearest neighbours," *African Journal of Computing & ICT*, vol. 16, no. 1, pp. 33–47, 2023.

[10] R. Venkataraman, K. Subramani, and P. Rajan, "A multi-module applicant tracking system: From job posting to employee hiring and performance monitoring," in *Proc. International Conference on Intelligent Systems and Control (ISCO)*, Coimbatore, India, 2024, pp. 1–7.

[11] Y. Zhang, H. Liu, X. Wang, and J. Chen, "Enhancing talent search ranking with role-aware expert mixtures and LLM-based fine-grained job descriptions," in *Proc. ACM SIGIR Conference on Research and Development in Information Retrieval*, Washington, DC, USA, 2024, pp. 2180–2190.

[12] A. Geyik, S. Ambler, and K. Kenthapadi, "Fairness-aware ranking in search & recommendation systems with application to LinkedIn talent search," in *Proc. ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, Anchorage, AK, USA, 2019, pp. 2221–2231.

[13] W. Park, J. Kim, and S. Lee, "AI-driven decision-making system for hiring process using multi-agent architecture and MCP tools," *arXiv preprint arXiv:2504.12345*, 2025.

[14] N. Agrawal, P. Jain, and S. Srivastava, "Optimizing job matching through automated resume screening with NLP and machine learning," in *Proc. International Conference on Artificial Intelligence and Smart Systems (ICAIS)*, Coimbatore, India, 2023, pp. 413–419.

[15] A. Mehrotra, V. Sahu, and R. Pandey, "ResumAI: Revolutionising automated resume analysis and recommendation with multi-model intelligence," *Journal of Artificial Intelligence Research*, vol. 78, pp. 1045–1078, 2023.

[16] H. Nguyen, S. Tran, and B. Pham, “Predicting skill shortages in labour markets: A machine learning approach using XGBoost and ANZSCO taxonomy,” *Computers & Industrial Engineering*, vol. 185, art. no. 109691, 2023.

[17] L. Pan, Y. Yao, and Y. Li, “A comprehensive survey of artificial intelligence techniques for talent analytics,” *ACM Computing Surveys*, vol. 56, no. 4, pp. 1–38, 2023.

[18] C. Hernandez, A. Lopez, and M. García, “Tec-Habilidad: Skill classification for bridging education and employment using GPT-3.5 and fine-tuned transformers,” in *Proc. European Conference on Information Retrieval (ECIR)*, Glasgow, UK, 2024, pp. 89–103.

[19] S. Kumar, R. Verma, and A. Nair, “Real-time AI-based skill gap analysis and adaptive career guidance using Gemini Flash and weighted cosine similarity,” in *Proc. International Conference on Advances in Computing, Communication and Control (ICAC3)*, Mumbai, India, 2024, pp. 1–8.

[20] G. Torres, F. Ruiz, and P. Sanchez, “Using graph algorithms for skills gap analysis: A Neo4j-based approach integrating PageRank, Louvain communities and O*NET ontology,” *Expert Systems with Applications*, vol. 238, art. no. 122156, 2024.